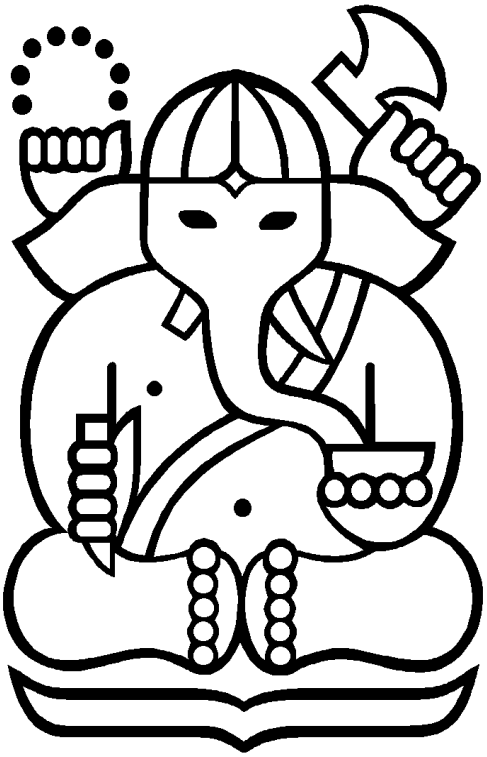
TUGAS BESAR 1: FEEDFORWARD NEURAL NETWORK

IF3270 PEMBELAJARAN MESIN



Disusun oleh:

Human earning - K01

Rafizan Muhammad Syawalazmi	13523034
Salman Hanif	13523056
Noumisyifa Nabila Nareswari	13523058

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG 2026

Daftar Isi

A. Deskripsi Persoalan.....3
B. Pembahasan.....3
a. Penjelasan Implementasi.....3
i. Deskripsi Kelas.....3
ii. Forward Propagation.....8
iii. Backward Propagation dan Weight Update.....9
b. Hasil Pengujian.....12
i. Pengaruh Depth.....12
iii. Pengaruh Fungsi Aktivasi.....14
iv. Pengaruh Learning Rate.....18
v. Pengaruh Inisialisasi Bobot.....23
vi. Pengaruh Regularisasi.....24
vii. Pengaruh Normalisasi RMSNorm.....30
viii. Perbandingan dengan Library sklearn.....30
C. Kesimpulan dan Saran.....31
D. Pembagian Tugas Tiap Anggota Kelompok.....31
E. Referensi.....32
F. Lampiran Form Penggunaan AI.....33

A. Deskripsi Persoalan

Persoalan utama dalam tugas ini adalah membangun sebuah modul FFNN beserta fungsi-fungsi terkait dari nol yang tepat secara teori dan memiliki performa – dari segi akurasi dan waktu – yang cukup mendekati *library* terkenal yang ada.

Adapun kriteria minimal yang harus diimplementasikan:

- Mampu menerima konfigurasi arsitektur FFNN yang fleksibel, termasuk penentuan jumlah *neuron* pada *input layer*, *hidden layer*, dan *output layer*.
- Mendukung berbagai fungsi aktivasi pada setiap *layer*, yaitu Linear, ReLU, Sigmoid, Hyperbolic Tangent (tanh), dan Softmax.
- Menyediakan opsi fungsi *loss* untuk evaluasi model yang mencakup MSE, Binary Cross-Entropy, dan Categorical Cross-Entropy.
- Memiliki mekanisme inisialisasi bobot dan bias secara Zero, Random Uniform, dan Random Normal dengan dukungan parameter *seed* untuk *reproducibility*.
- Mampu menjalankan *forward propagation* dan *backward propagation* untuk *input* data dalam bentuk *batch* dan menggunakan *chain rule* untuk perhitungan gradien.
- Mengimplementasikan metode pembaruan bobot dengan *Gradient Descent*.
- Mengimplementasikan metode regularisasi L1 serta L2.

Selain membangun modul FFNN beserta berbagai fungsi terkait, persoalan juga mencakup pengujian pada modul yang telah dibuat untuk melakukan perbandingan performa dengan *library* terkenal yang ada sekaligus eksplorasi *hyperparameter* (parameter yang akan dieksplorasi: *depth*, *width*, *learning rate*, fungsi aktivasi, fungsi inisialisasi, metode regularisasi, dan normalisasi). Pengujian dilakukan menggunakan dataset 'Global Student Placement and Salary' dan model akan mempredikisi kelas 'placement_status' yang dapat bernilai *placed* atau *not placed*.

B. Pembahasan

a. Penjelasan Implementasi

i. Deskripsi Kelas

Untuk mendapatkan gambaran yang lebih jelas terhadap struktur yang ada, berikut *directory tree* dari *repository* kami.

```
.
├── doc/
├── src/
│   ├── ffnn/
│   │   ├── __pycache__/
```

```
│   │   ├── __init__.py
│   │   ├── activations.py
│   │   ├── autodiff.py
│   │   ├── ffnn.py
│   │   ├── layer.py
│   │   ├── losses.py
│   │   ├── main_testing.py
│   │   └── utils.py
│   ├── datasetml_2026.csv
│   └── testing.ipynb
└── README.md
```

Source code yang berisikan implementasi operasi matematika hanya berisikan berbagai implementasi fungsi, tidak dalam bentuk kelas. *Source code* tersebut di antaranya adalah *activations.py*, *utils*.py, *losses.py*, *testing.py*. Namun, untuk komponen yang perlu *stateful*, seperti modul FFNN itu sendiri, terkait *autodifferentiation*, dan terkait *layer*, dibangun dengan konsep *Object Oriented Programming* (OOP) dalam bentuk kelas. Berikut penjabaran kelas-kelas yang ada.

- **Kelas FFNN**

  Ini adalah struktur utama dari model FFNN yang telah dibangun. Di sini, semua modul lain dipanggil dan fungsi-fungsinya digunakan untuk membangun satu kesatuan kelas FFNN.

  Atribut

  Semua parameter untuk model FFNN menjadi bagian dari atribut kelas FFNN ini, selain itu layer_sizes, activation functions, dan init method juga menjadi atribut. Secara lengkap, semua atribut dari kelas FFNN ini ditulis di bawah ini :

  - **layer_sizes** : list of int = berisi informasi jumlah nodes dan jumlah layer. cth input : [3, 16, 8, 2] artinya 3 *nodes input layer*, 16 *nodes hidden layer* 1, 8 *nodes hidden layer* 2, dan 3 *nodes output layer*.
  - **activations** : *list of str* = berisi informasi fungsi aktivasi dari setiap transisi *layer*. cth input : [‘relu’, ‘sigmoid’, ‘linear’] artinya setiap transisi *layer* menggunakan fungsi aktivasi tertulis secara berurutan.
  - **init_method** : *str* = metode yang dipilih untuk inisialisasi nilai *Weight* dan bias dari setiap transisi layer. Terdapat 5 metode, yaitu : *zero*, *random uniform*, *random normal*, xavier, dan he.
  - **learning_rate** : float = besaran variabel yang memengaruhi ukuran langkah (step size) saat memperbarui bobot *updateWeights* saat proses optimasi. Menentukan seberapa cepat model berkonvergensi menuju titik minimum loss.

- **epochs** : int = Jumlah total iterasi di mana seluruh dataset pelatihan dilewatkan ke dalam model. Pada setiap awal epoch, data di-*shuffle* untuk memastikan model mempelajari pola fitur secara umum dan tidak terpengaruh oleh urutan masuknya data
- **batch_size** : int = Jumlah sampel data yang diproses dalam satu kali *forward* dan *backward pass* sebelum bobot model diperbaru
- **l1_lambda :** float = Koefisien penalti untuk Regularisasi L1 (Lasso). Teknik ini menambahkan nilai absolut dari bobot ke dalam fungsi *loss*, yang cenderung memaksa beberapa bobot menjadi nol (mendorong *sparsity*) guna melakukan seleksi fitur secara otomatis
- **l2_lambda :** float = Koefisien penalti untuk Regularisasi L2 (Ridge/Weight Decay). Teknik ini menambahkan nilai kuadrat dari bobot ke dalam fungsi *loss* untuk mencegah nilai bobot menjadi terlalu ekstrem
- **verbose** : boolean = Parameter kontrol untuk menentukan apakah model akan menampilkan log kemajuan pelatihan ke terminal secara *real-time* (seperti *progress bar*, nilai *loss*, dan akurasi tiap epoch) atau tidak
- **** init_params :** Kumpulan argumen kata kunci (keyword arguments) tambahan yang digunakan secara fleksibel untuk mengonfigurasi parameter inisialisasi bobot, seperti nilai mean, std, seed, atau konstanta epsilon pada RMS Norm.

Method

Berikut berbagai method pada kelas FFNN :

- **__init__** : Konstruktor utama untuk membangun arsitektur jaringan saraf. Method ini bertugas menginisialisasi parameter global model, memvalidasi kesesuaian antara jumlah layer dan fungsi aktivasi, serta membangun objek-objek Layer berdasarkan spesifikasi yang diberikan dalam layer_sizes.
- **forward(X)** : Menjalankan proses perambatan maju (Forward Propagation) di mana data input X dialirkan melalui setiap layer di dalam jaringan secara berurutan. Hasil akhir dari method ini adalah objek Node yang menampung probabilitas prediksi serta riwayat komputasi untuk keperluan autodiff.
- **predict(X)** : Method khusus untuk fase inferensi. Fungsi ini memanggil forward(X) namun hanya mengembalikan data dalam bentuk array NumPy mentah
- **getParameters()**: Mengumpulkan semua parameter yang dapat dipelajari (objek Node dari bobot W dan bias b) dari seluruh layer yang ada

- **applyRegularization()**: Menghitung total nilai penalti dari teknik regularisasi L1 dan L2 pada setiap matriks bobot
- **updateWeights**(): Mengimplementasikan algoritma optimasi (seperti SGD) untuk memperbarui nilai data pada setiap parameter berdasarkan gradien yang telah dihitung. Setelah pembaruan selesai, method ini akan mereset semua gradien menjadi nol untuk persiapan iterasi berikutnya.
- **_calculate_accuracy**(y_true, y_pred_probs): Sebuah method internal (helper) yang menghitung akurasi model dengan membandingkan label target asli dengan hasil prediksi
- **fit**(X_train, y_train, X_val, y_val): Jantung dari proses pelatihan model. Method ini mengoordinasikan siklus training yang meliputi pengacakan data (shuffling), pemrosesan per batch, eksekusi forward-backward pass, penghitungan loss, hingga pencatatan riwayat (history) performa model untuk setiap epoch
- **display_weight_distribution**(target_layers): Menghasilkan laporan statistik deskriptif di terminal mengenai distribusi nilai bobot (W) dan bias (b). Laporan mencakup nilai minimum, maksimum, rata-rata, dan standar deviasi untuk layer tertentu
- **display_gradient_distribution**(target_layers): Serupa dengan distribusi bobot, method ini menampilkan statistik gradien (dW dan db) yang dihasilkan setelah proses backward
- **saveModel**(filepath): Melakukan serialisasi terhadap arsitektur dan bobot model yang telah dilatih menggunakan modul pickle.
- **loadModel**(filepath): Sebuah class method yang digunakan untuk membangun kembali instansi kelas FFNN dari file yang telah disimpan sebelumnya.

- **Kelas Layer**

  Seperti namanya, setiap transisi layer informasi weight, bias, jumlah node di layer masuk, dan jumlah node di layer tujuan, beserta fungsi aktivasi terdefinisi lengkap.

  Atribut

  - **activation_name** : string = menyimpan nama fungsi aktivasi yang digunakan oleh layer
  - **use_rms_norm :** boolean = *toggle* yang menentukan apakah layer ini akan menerapkan teknik Root Mean Square Normalization sebelum data masuk ke fungsi aktivasi

- **W :** node = Matriks bobot (*weights*) layer yang telah dibungkus ke dalam objek Node. Objek ini menyimpan data bobot dalam bentuk array NumPy serta gradien yang dihasilkan selama proses *backpropagation*
- **b :** node = Vektor bias layer dalam bentuk objek Node

Method

- **__init__ :** Konstruktor kelas yang bertanggung jawab untuk menetapkan dimensi layer, memilih skema inisialisasi bobot (seperti He atau Xavier), dan menginisialisasi parameter W serta b
- **forward(X) :** Melakukan operasi matematika inti pada layer. Proses ini meliputi:
  1. Operasi Linear: Melakukan perkalian matriks antara input X dan bobot W, kemudian menambahkannya dengan bias b (Z = X . W + b).
  2. Normalisasi: Menerapkan rms_norm() jika atribut use_rms_norm bernilai True.
  3. Aktivasi: Menerapkan fungsi non-linear (seperti ReLU atau Softmax) pada hasil komputasi sebelum diteruskan ke layer berikutnya.
- **getParameters() :** Method sederhana yang mengembalikan list berisi objek Node dari bobot (W) dan bias (b)

- **Kelas Tensor / Node**

  komponen paling fundamental dalam sistem *Automatic Differentiation* (AutoDiff)

  Atribut

  - **data :** np.ndarray = menyimpan nilai numerik utama (tensor/matriks) dari variabel tersebut. Atribut ini digunakan dalam semua perhitungan *forward pass*
  - **grad :** np.ndarray = Menyimpan nilai gradien (turunan) terhadap *loss*. Nilai ini mengakumulasi instruksi koreksi yang dihitung selama fase *backpropagation*.
  - **_backward :** function = Sebuah fungsi internal (biasanya *lambda* atau fungsi bersarang) yang menyimpan rumus turunan lokal dari operasi matematika yang menghasilkan *node* tersebut
  - **_prev :** set = Kumpulan *node* induk (*parent nodes*) yang terlibat dalam operasi pembuatan *node* ini. Atribut ini membentuk silsilah grafik komputasi.

  Method

  1. **Operasi Aritmatika (overloading)**

- **__add__ (other) :** Method ini mendefinisikan bahwa gradien dari hasil penjumlahan akan didistribusikan secara merata ke kedua inputnya
- **__matmul__ (other) :** Mengimplementasikan perkalian matriks (operator @). Method ini menggunakan aturan rantai (chain rule) matriks yang melibatkan operasi transpose untuk menghitung gradien bagi kedua matriks input

2. **Method Aktivasi & Normalisasi**
   - **linear(), relu(), sigmoid(), tanh(), softmax(), swish(), leaky_relu():** Kumpulan method ini menerapkan fungsi non-linearitas pada data. Masing-masing method menempelkan fungsi _backward spesifik yang berisi turunan matematis dari masing-masing fungsi aktivasi tersebut
   - **rms_norm(epsilon)**: Melakukan normalisasi data berdasarkan nilai Root Mean Square
3. **Method Kontrol Autodiff**
   - **backward() :** Method pemicu utama untuk memulai kalkulus otomatis. Method ini melakukan : Topological sort (mengurutkan semua node dalam grafik komputasi dari akhir ke awal), Gradient Initialization (memberikan nilai awal 1.0 pada gradien *loss),* Recursive Execution (menjalankan setiap fungsi _backward secara berurutan munduk untuk mengisi semua nilai gradien dalam jaringan).

ii. Forward Propagation

Forward Propagation adalah proses di mana data input dialirkan melalui neural network dari lapisan input ke lapisan output. Dalam proyek ini, setiap tahap komputasi direkam oleh kelas Node (atau Tensor) untuk membangun grafik komputasi yang nantinya digunakan pada tahap Backpropagation.

- **Alur Matematika per Layer**

Pada setiap layer di kelas Layer, terjadi dua transformasi utama sebelum data diteruskan ke layer berikutnya:

a. operasi linear

Pada setiap layer di kelas Layer, nput dari layer sebelumnya (X) dikalikan dengan matriks bobot (W) dan ditambahkan dengan bias (b). Secara matematis diimplementasikan dalam kode sebagai:

iii. *Backward Propagation* dan *Weight Update*

Backward propagation diimplementasikan menggunakan konsep *automatic differentiation* berbasis *topological sort*. Setiap operasi matematis membangun graf yang kemudian *traversed* secara terbalik untuk menghitung gradien. Berikut atribut-atribut yang ada pada kelas Node

```
class Node:
    def __init__(self, data, _children=()):
        # Inisialisasi data dan gradien dalam np
        if not isinstance(data, np.ndarray):
            self.data = np.array(data, dtype=np.float64)
        else: self.data = data

        self.grad = np.zeros_like(self.data,
dtype=np.float64)
        self._backward = lambda: None
        self._prev = set(_children)
```

'grad' menyimpan gradien *note* tersebut terhadap *loss* dan diinisialisasi dengan nilai 0. '_backward' adalah fungsi yang mendefinisikan bagaimana gradien diteruskan ke *node-node* pembentuknya. Lalu, '_prev' melacak *parent node* untuk membangun graf.

Setiap operasi matematis punya definisi '_backward'-nya sendiri

Penjumlahan:

```
def __add__(self, other):
    if not isinstance(other, Node):
        other = Node(other)
    else:
        other = other

    out = Node(self.data + other.data, (self, other))

    def _backward():
        # gradient untuk self
        self.grad += out.grad
        # gradient untuk other
        if self.data.shape != other.data.shape:
            other.grad += np.sum(out.grad, axis=0,
keepdims=True)
        else:
            other.grad += out.grad

    out._backward = _backward
    return out
```

Perkalian matriks:

```
def __matmul__(self, other):
```

```
    other = other if isinstance(other, Node) else Node(other)
    out = Node(self.data @ other.data, (self, other))

    def _backward():
        # Aturan rantai untuk perkalian matriks memerlukan
Transpose (.T)
        self.grad += out.grad @ other.data.T
        other.grad += self.data.T @ out.grad

    out._backward = _backward
    return out
```

Lalu, untuk fungsi aktivasi, semuanya punya pola implementasi yang serupa. Intinya, kalikan turunan fungsi aktivasinya dengan gradien masuk. Berikut contoh implementasinya untuk ReLU. Pola yang sama digunakan juga untuk fungsi aktivasi lainnya seperti sigmoid, tanh, softmax, dll.

```
def relu(self):
    out = Node(act.relu(self.data), (self,))
    def _backward():
        self.grad += act.d_relu(self.data) * out.grad
    out._backward = _backward
    return out
```

Selanjutnya, proses *backward* keseluruhan dijalankan melalui *method* 'backward()':

```
def backward(self):
    # Bangun urutan eksekusi mundur
    topo = []
    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)

    build_topo(self)

    # Set gradien output akhir menjadi 1 (Pemicu awal rantai
kalkulus)
    self.grad = np.ones_like(self.data, dtype=np.float64)

    # Jalankan _backward() satu per satu dari ujung output
mundur ke input
    for v in reversed(topo):
        v._backward()
```

Alur proses yang terjadi adalah sebagai berikut:

- Bangun urutan topologis seluruh *graf* secara rekursif dari *output node*

- *Set* variabel *gradien node output* dengan nilai 1 sebagai pemicu awal *chain rule*.
- Jalankan '_backward()' dari *output* secara mundur hingga ke input, memastikan setiap *node* sudah menerima gradien lengkap sebelum meneruskannya ke bawah.

Untuk gradien *loss*, di dalam 'losses.py' dalam tiap *loss function*-nya juga didefinisikan '_backward' sesuai turunan masing-masing fungsi *loss*.

```
# Turunan dari MSE
def _backward():
    y_pred.grad += (2.0 / N) * diff * out.grad # Turunan
matematis d(MSE) / d(y_pred) = 2/N * (y_pred - y_true)

# Turunan dari CCE
def _backward():
    if from_softmax:
        grad = (y_pred.data - y_true) / batch_size
        y_pred.grad += grad * out.grad
    else:
        grad = (-y_true / y_pred_clipped) / batch_size
        y_pred.grad += grad * out.grad

# Turunan dari BCE
def _backward():
    grad = (-(y_true / y_pred_clipped) + ((1 - y_true) / (1 -
y_pred_clipped))) / batch_size
    y_pred.grad += grad * out.grad
```

Setelah gradien terisi seluruhnya, dilakukan proses *weight update* dalam kelas FFNN

```
loss_node.backward()
reg_l = self.applyRegularization()
self.updateWeights()
```

Alur yang terjadi adalah:

- 'backward()' mengisi '.grad' di seluruh *weight* dan *bias* lewat *autodiff*.
- Menambahkan gradien penalti langsung ke '.grad' yang sudah ada lewat pemanggilan'applyRegularization()'
- Memanggil 'updateWeights()' yang akan menggunakan gradien tersebut untuk memperbarui bobot

Lalu, algoritma yang dieksekusi oleh ‘updateWeights()’ tergantung dengan *optimizer* yang digunakan saat pengguna *initialize* objek FFNN baru. Jika *optimizer* yang dipilih adalah Stochastic Gradient Descent, maka yang dieksekusi adalah:

```
for p in params:
    p.data -= self.learning_rate * p.grad
    p.grad = np.zeros_like(p.data)
```

Bobot diperbarui secara langsung, lalu gradien di-*reset* ke 0 untuk *batch* berikutnya. Lalu, jika pengguna memilih Adam, yang akan dieksekusi adalah sebagai berikut.

```
self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * p.grad
self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) * (p.grad ** 2)
m_hat = self.m[i] / (1 - self.beta1 ** self.t)
v_hat = self.v[i] / (1 - self.beta2 ** self.t)
p.data -= self.lr * m_hat / (np.sqrt(v_hat) + self.epsilon)
```

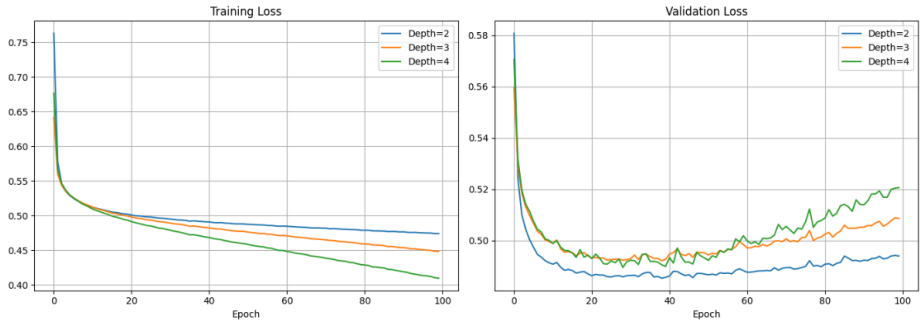
Adam menyimpan dua *moving average* yaitu m untuk momentum (rata-rata gradien) dan v untuk rata-rata gradien kuadrat. Keduanya dikoreksi bias karena di awal *training* nilainya cenderung mendekati 0. Update akhir melakukan pembagian momentum dengan akar v sehingga *learning rate* efektif menyesuaikan tiap parameter, berbeda dengan SGD yang menggunakan *learning rate* seragam untuk semua parameter.

b. Hasil Pengujian

Berikut adalah hasil pengujian dengan beberapa variabel bebas yang berbeda.

i. Pengaruh *Depth*

Pengujian dilakukan dengan 3 variasi *Depth*, yaitu 2, 3, dan 4. Pengujian dilakukan menggunakan arsitektur [11, 64, X, 2] dengan kombinasi fungsi aktivasi ReLU untuk hidden layer dan softmax untuk output layer, inisiasi He, 100 epoch, learning rate 0.01, dan *batch size* 32.



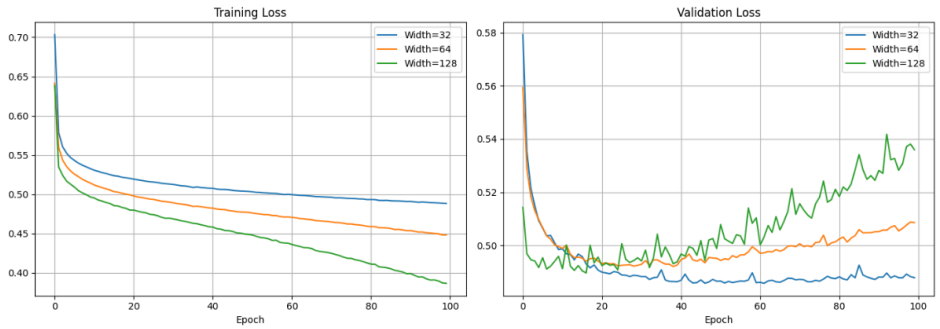
Seperti yang dapat dilihat pada grafik *loss* di atas, Depth=4 konvergen paling cepat dan mencapai training loss terendah (~0.41), namun validation lossnya mulai naik secara bertahap setelah epoch 30-an hingga mencapai titik tertingginya di akhir epoch yang mengindikasikan terjadinya overfitting. Depth=2 menunjukkan konvergensi yang paling stabil; meskipun training lossnya paling tinggi, model ini berhasil mencapai dan mempertahankan validation loss terendah dengan gap training dan validation loss yang kecil. Depth=3 berada di antara keduanya, dengan training loss yang lebih rendah dari Depth=2 namun validation loss-nya mulai menunjukkan tren kenaikan setelah epoch 50.

```
Depth=2 | Test Accuracy: 0.7470
Depth=3 | Test Accuracy: 0.7420
Depth=4 | Test Accuracy: 0.7330
```

Depth=2 menghasilkan akurasi tes tertinggi sebesar 74.70%. Depth=4 memiliki kompleksitas yang terlalu besar sehingga cenderung mengalami overfitting, menghasilkan akurasi tes terendah sebesar 73.30%. Depth=3 performanya cukup baik 74.20% namun masih di bawah Depth=2.

ii. Pengaruh *Width*

Pengujian dilakukan dengan 3 variasi *Width*, yaitu 32, 64, dan 128. Pengujian dilakukan menggunakan arsitektur [11, X, 3, 2] dengan kombinasi fungsi aktivasi ReLU-ReLU-Softmax, inisiasi He, 100 epoch, learning rate 0.01, dan *batch size* 32.



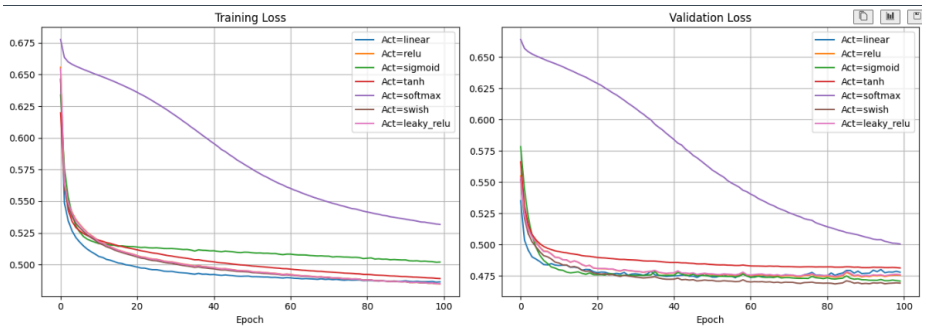
Seperti yang dapat dilihat pada grafik *loss* di atas, Width=128 konvergen paling cepat dan mencapai training loss terendah, namun validation loss-nya naik secara signifikan dan sangat fluktuatif setelah epoch 20 hingga mencapai 0.54, mengindikasikan terjadinya overfitting yang jelas. Width=32 menunjukkan konvergensi yang stabil dengan gap training dan validation loss yang kecil dan paling konsisten tanpa menunjukkan tanda-tanda kenaikan hingga akhir epoch. Width=64 menunjukkan performa di antara keduanya.

```
Width=32 | Test Accuracy: 0.7435
Width=64 | Test Accuracy: 0.7420
Width=128 | Test Accuracy: 0.7335
```

Width=32 menghasilkan akurasi tertinggi sebesar 74.35%. Width=128 terlalu kompleks dan memiliki kapasitas parameter yang terlalu besar, sehingga kemungkinan besar mengalami overfitting pada data latih yang menghasilkan akurasi tes terendah sebesar 73.35%. Width=64 performanya cukup baik sebesar 74.20% dan sangat mendekati Width=32.

iii. Pengaruh Fungsi Aktivasi

Pengujian dilakukan dengan 7 variasi Fungsi Aktivasi yaitu linear, ReLU, sigmoid, tanh, softmax, swish, dan leaky ReLU hanya pada hidden layer pertama. Pengujian dilakukan menggunakan arsitektur [11, 64, 32, 2] dengan kombinasi fungsi aktivasi X-ReLU-Softmax, inisiasi He, 100 epoch, learning rate 0.01, dan *batch size* 32.

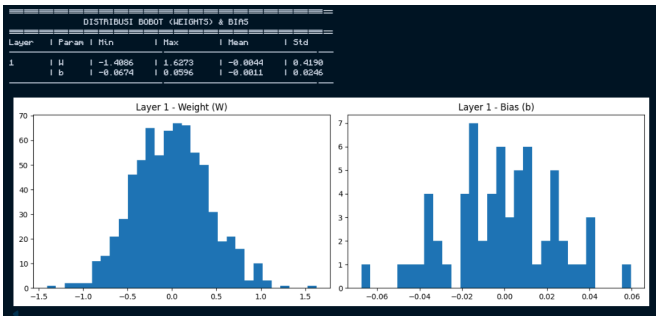
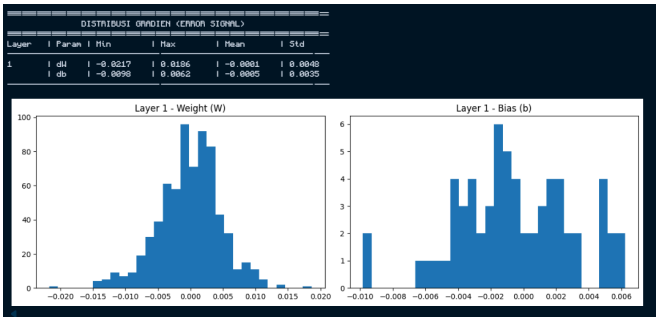


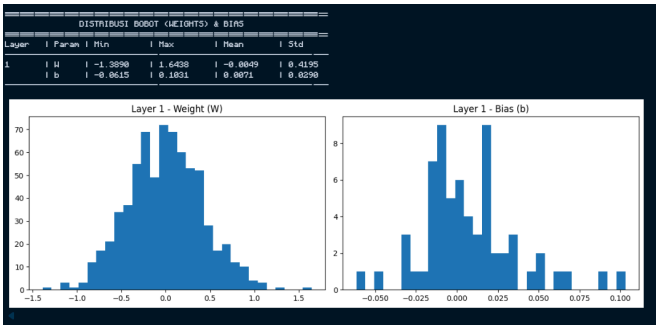
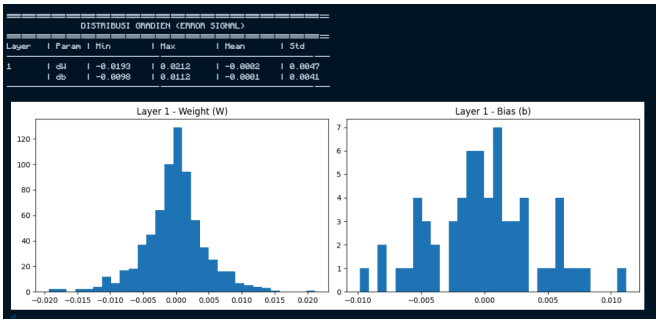
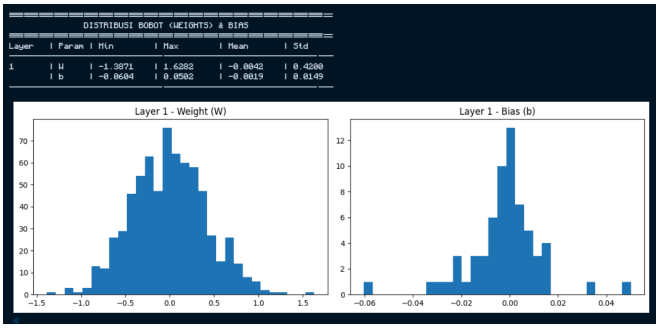
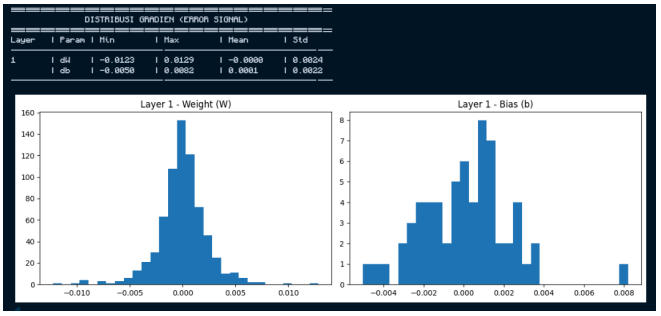
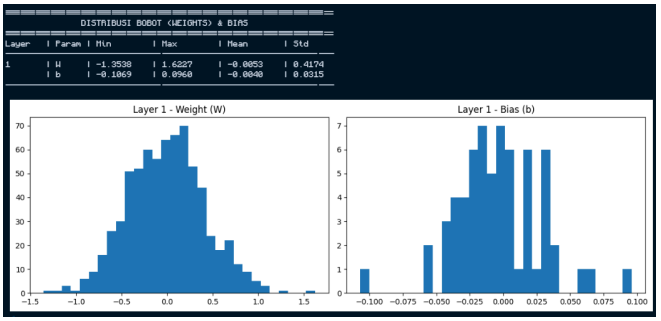
Seperti yang dapat dilihat pada grafik *loss* di atas, Act=linear konvergen sangat cepat di awal dan mencapai training loss terendah (~0.48), namun validation loss-nya cenderung stagnan lebih awal dan sedikit berfluktuasi. Act=swish, relu, dan leaky_relu menunjukkan konvergensi yang paling stabil dan optimal, berhasil mencapai serta mempertahankan validation loss terendah (~0.47) tanpa menunjukkan tanda-tanda overfitting. Sebaliknya, Act=softmax konvergen paling lambat secara signifikan dengan nilai loss yang masih terus menurun tajam di akhir grafik, mengindikasikan bahwa

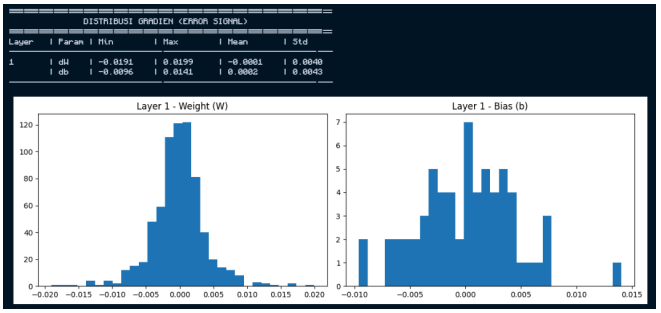
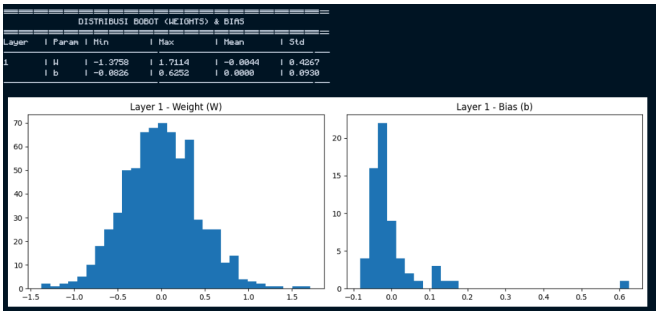
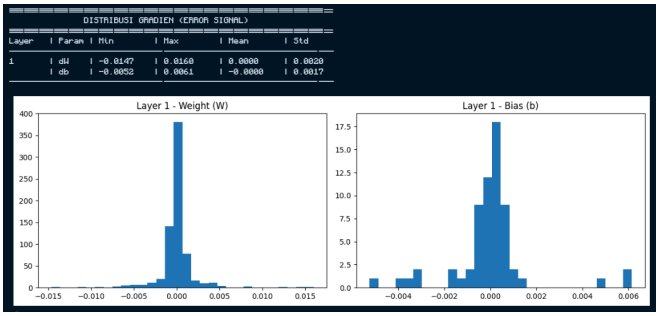
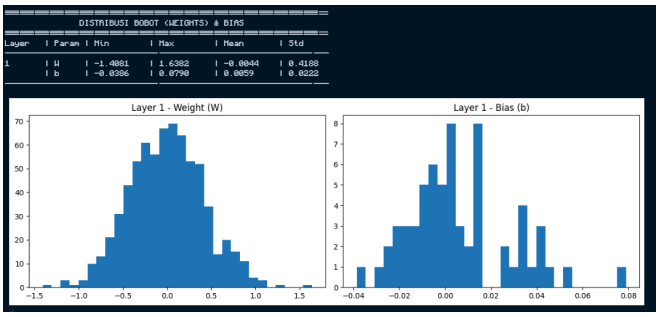
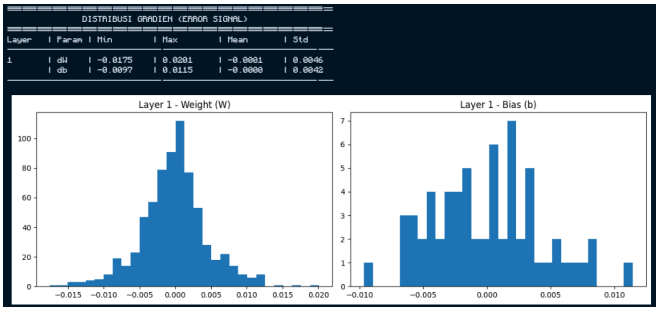
model belum konvergen sepenuhnya dalam 100 epoch. Selain itu, Act=tanh dan sigmoid memiliki loss yang tidak jauh berbeda dengan mayoritas fungsi aktivasi lainnya.

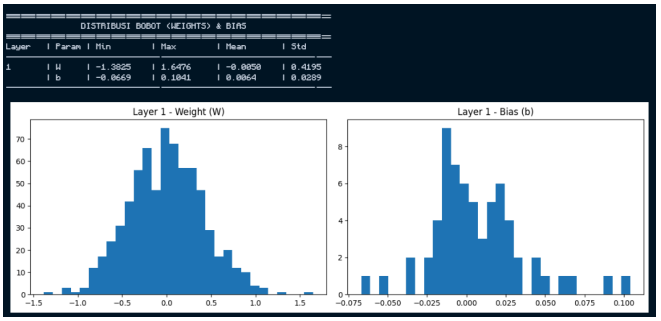
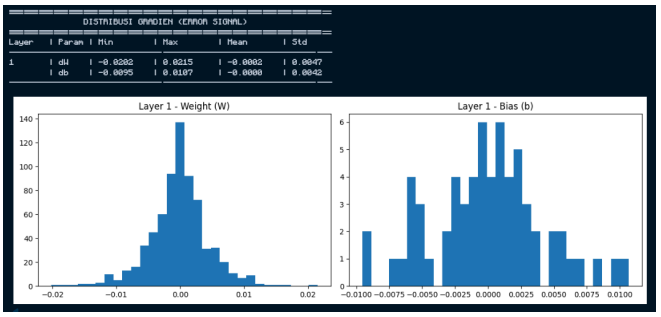
```
First Layer Activation=linear | Test Accuracy: 0.7585
First Layer Activation=relu | Test Accuracy: 0.7520
First Layer Activation=sigmoid | Test Accuracy: 0.7570
First Layer Activation=tanh | Test Accuracy: 0.7440
First Layer Activation=softmax | Test Accuracy: 0.7375
First Layer Activation=swish | Test Accuracy: 0.7530
First Layer Activation=leaky_relu | Test Accuracy: 0.7510
```

Act=linear menghasilkan akurasi tes tertinggi (75.85%). Act=softmax kurang tepat jika digunakan sebagai fungsi aktivasi pada hidden layer karena sifatnya yang menormalisasi distribusi nilai (menjumlahkan output menjadi 1), sehingga membatasi kapasitas representasi pembelajaran model dan menghasilkan akurasi terendah (73.75%). Fungsi aktivasi non-linear lainnya seperti sigmoid, swish, dan relu performanya cukup baik dan konsisten (berkisar antara 75.20% hingga 75.70%), menunjukkan bahwa fungsi-fungsi standar ini bekerja dengan stabil dan optimal dalam menggeneralisasi data.

Fungsi Aktivasi	**Komponen Pengujian**	**Hasil Uji**
linear	Distribusi bobot dan bias	
	Distribusi gradien	

ReLU	Distribusi bobot dan bias	
	Distribusi gradien	
sigmoid	Distribusi bobot dan bias	
	Distribusi gradien	
tanh	Distribusi bobot dan bias	

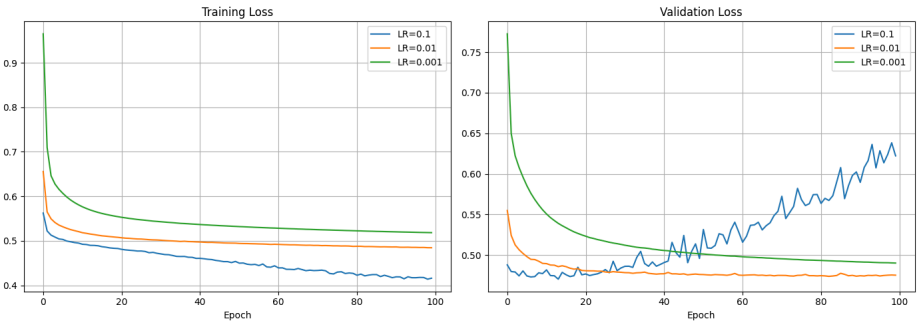
	Distribusi gradien	
softmax	Distribusi bobot dan bias	
	Distribusi gradien	
swish	Distribusi bobot dan bias	
	Distribusi gradien	

leaky ReLU	Distribusi bobot dan bias	
	Distribusi gradien	

Untuk distribusi bobot, semua fungsi aktivasi memiliki distribusi yang mirip, yaitu menyerupai bentuk normal. Untuk distribusi bias, fungsi aktivasi masing-masing memiliki perbedaan yang cukup signifikan.

iv. Pengaruh *Learning Rate*

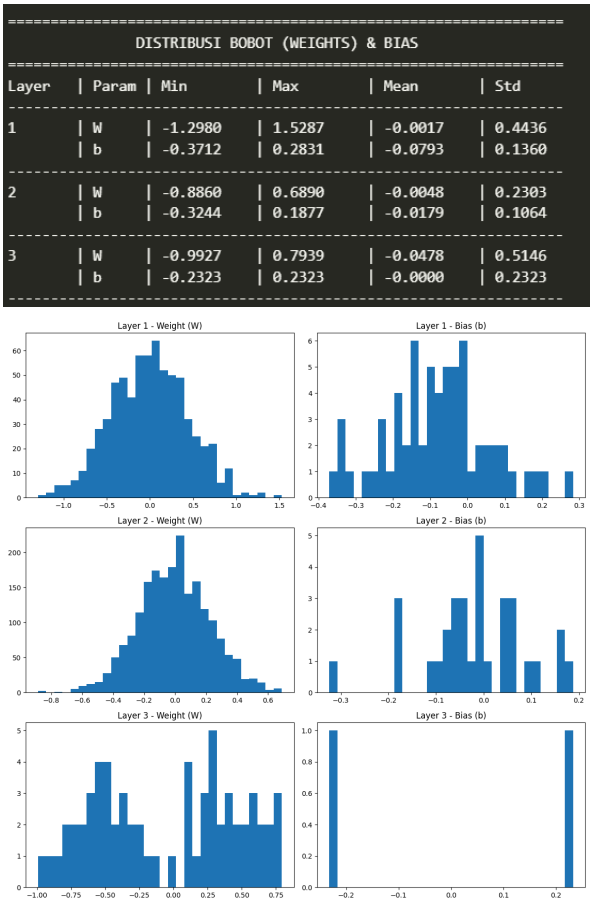
Pengujian dilakukan dengan 3 variasi *learning rate*, yaitu 0.1, 0.01, dan 0.001. Pengujian dilakukan menggunakan arsitektur tetap [11, 64, 32, 2] dengan kombinasi fungsi aktivasi ReLU-ReLU-Softmax, inisialisasi dengan metode He, 100 epoch, dan *batch size* 32.



Seperti yang dapat dilihat pada grafik *loss* di atas, LR=0.1 konvergen paling cepat dengan training loss terendah (~0.41), namun *validation loss* naik secara signifikan, mengindikasikan terjadinya *overfitting*. Lalu, LR=0.01 menunjukkan konvergensi stabil dengan *gap* antara *training* dan *validation loss* yang kecil dan konsisten. LR=0.001 mencapai konvergen paling lambat dan belum sepenuhnya konvergen dalam 100 *epoch*.

```
LR=  0.1 | Test Accuracy: 0.7210
LR= 0.01 | Test Accuracy: 0.7520
LR=0.001 | Test Accuracy: 0.7455
```

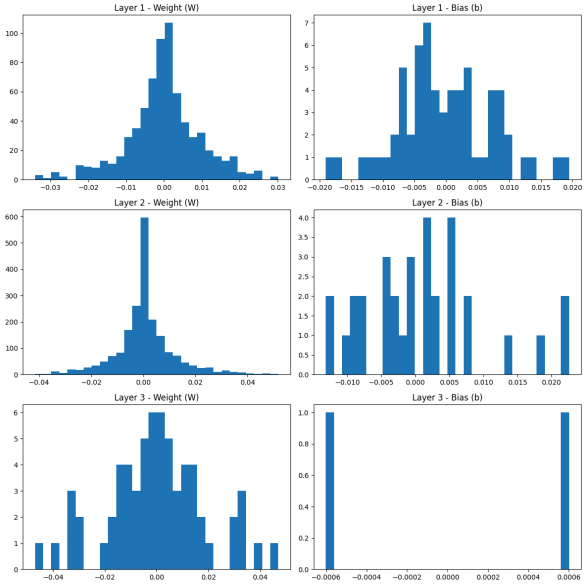
LR=0.01 menghasilkan akurasi tertinggi (75.20%), diikuti LR=0.001 (74.55%), dan LR=0.1 (72.10%). LR=0.1 *underperform* meskipun *training loss*-nya rendah. Kemungkinan besar hal ini disebabkan oleh terjadinya *overfitting* seperti yang terbaca pada grafik sebelumnya.

Learning Rate	**Komponen Pengujian**	**Hasil Uji**
0.1	Distribusi bobot dan bias	DISTRIBUSI BOBOT (WEIGHTS) & BIAS 

```
================================================================
              DISTRIBUSI BOBOT (WEIGHTS) & BIAS
================================================================
Layer   | Param | Min        | Max        | Mean       | Std
----------------------------------------------------------------
1       | W     | -1.2980    | 1.5287     | -0.0017    | 0.4436
        | b     | -0.3712    | 0.2831     | -0.0793    | 0.1360
----------------------------------------------------------------
2       | W     | -0.8860    | 0.6890     | -0.0048    | 0.2303
        | b     | -0.3244    | 0.1877     | -0.0179    | 0.1064
----------------------------------------------------------------
3       | W     | -0.9927    | 0.7939     | -0.0478    | 0.5146
        | b     | -0.2323    | 0.2323     | -0.0000    | 0.2323
----------------------------------------------------------------
```

<table>
<tr>
<td></td>
<td>Distribusi gradien</td>
<td>

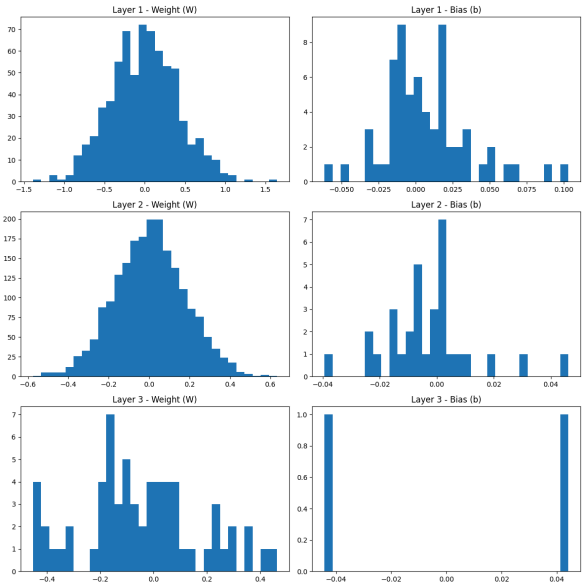
```
================================================================
                DISTRIBUSI GRADIEN (ERROR SIGNAL)
================================================================
Layer   | Param | Min        | Max        | Mean       | Std
----------------------------------------------------------------
1       | dW    | -0.0341    | 0.0302     | -0.0000    | 0.0097
        | db    | -0.0190    | 0.0195     | -0.0002    | 0.0075
----------------------------------------------------------------
2       | dW    | -0.0416    | 0.0518     | 0.0007     | 0.0113
        | db    | -0.0132    | 0.0226     | 0.0010     | 0.0089
----------------------------------------------------------------
3       | dW    | -0.0470    | 0.0470     | 0.0000     | 0.0186
        | db    | -0.0006    | 0.0006     | -0.0000    | 0.0006
----------------------------------------------------------------
```



</td>
</tr>
<tr>
<td>0.01</td>
<td>Distribusi bobot dan bias</td>
<td>

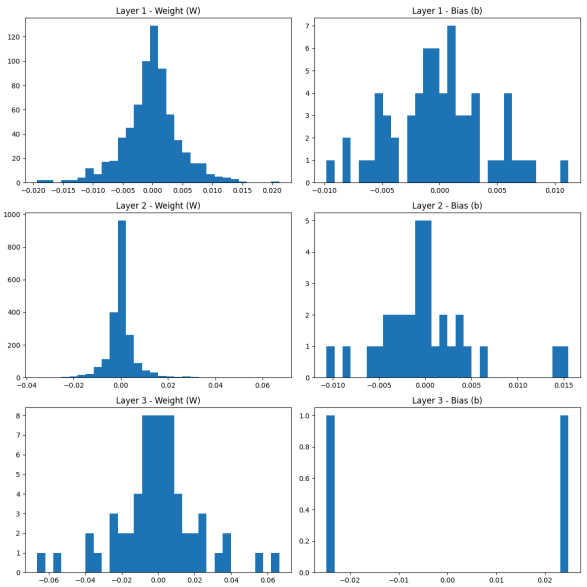
```
================================================================
                DISTRIBUSI BOBOT (WEIGHTS) & BIAS
================================================================
Layer   | Param | Min        | Max        | Mean       | Std
----------------------------------------------------------------
1       | W     | -1.3890    | 1.6438     | -0.0049    | 0.4195
        | b     | -0.0615    | 0.1031     | 0.0071     | 0.0290
----------------------------------------------------------------
2       | W     | -0.5752    | 0.6328     | 0.0058     | 0.1749
        | b     | -0.0395    | 0.0461     | -0.0032    | 0.0159
----------------------------------------------------------------
3       | W     | -0.4518    | 0.4629     | -0.0478    | 0.2186
        | b     | -0.0442    | 0.0442     | -0.0000    | 0.0442
----------------------------------------------------------------
```

</td>
</tr>
</table>



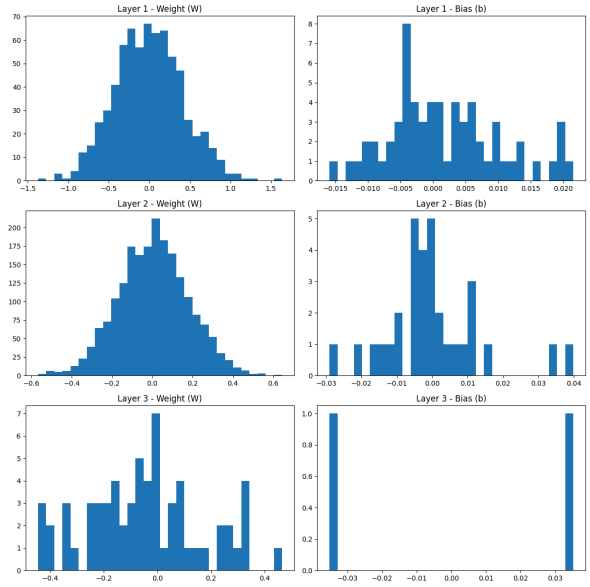
Distribusi gradien

```
==============================================================
                DISTRIBUSI GRADIEN (ERROR SIGNAL)
==============================================================
Layer   | Param | Min        | Max        | Mean       | Std
--------------------------------------------------------------
1       | dW    | -0.0193    | 0.0212     | -0.0002    | 0.0047
        | db    | -0.0098    | 0.0112     | -0.0001    | 0.0041
--------------------------------------------------------------
2       | dW    | -0.0354    | 0.0671     | 0.0001     | 0.0062
        | db    | -0.0107    | 0.0155     | 0.0000     | 0.0052
--------------------------------------------------------------
3       | dW    | -0.0663    | 0.0663     | 0.0000     | 0.0220
        | db    | -0.0248    | 0.0248     | -0.0000    | 0.0248
--------------------------------------------------------------
```



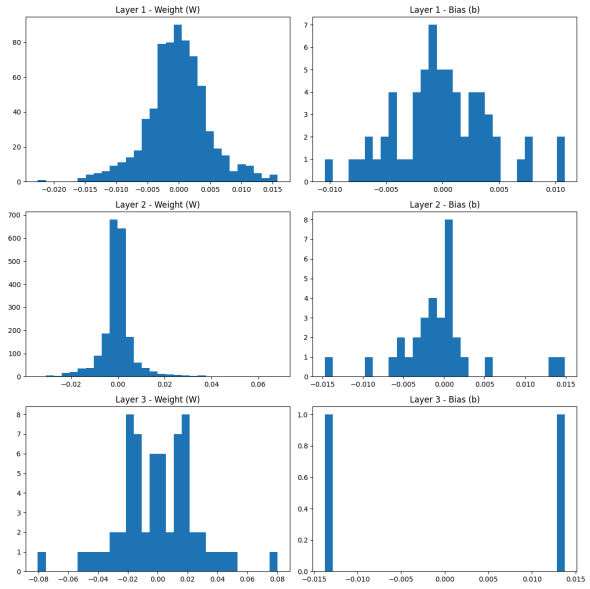
<table>
<tr>
<td rowspan="2">0.001</td>
<td>Distribusi bobot dan bias</td>
<td>

```
================================================================
              DISTRIBUSI BOBOT (WEIGHTS) & BIAS
================================================================
Layer   | Param | Min        | Max        | Mean       | Std
----------------------------------------------------------------
1       | W     | -1.3731    | 1.6344     | -0.0045    | 0.4193
        | b     | -0.0159    | 0.0215     | 0.0017     | 0.0086
----------------------------------------------------------------
2       | W     | -0.5667    | 0.6435     | 0.0071     | 0.1745
        | b     | -0.0291    | 0.0399     | -0.0000    | 0.0135
----------------------------------------------------------------
3       | W     | -0.4448    | 0.4638     | -0.0478    | 0.2102
        | b     | -0.0353    | 0.0353     | -0.0000    | 0.0353
----------------------------------------------------------------
```



</td>
</tr>
<tr>
<td>Distribusi gradien</td>
<td>

```
================================================================
              DISTRIBUSI GRADIEN (ERROR SIGNAL)
================================================================
Layer   | Param | Min        | Max        | Mean       | Std
----------------------------------------------------------------
1       | dW    | -0.0225    | 0.0158     | -0.0002    | 0.0050
        | db    | -0.0105    | 0.0108     | -0.0000    | 0.0042
----------------------------------------------------------------
2       | dW    | -0.0343    | 0.0684     | -0.0003    | 0.0071
        | db    | -0.0147    | 0.0149     | -0.0006    | 0.0053
----------------------------------------------------------------
3       | dW    | -0.0805    | 0.0805     | 0.0000     | 0.0257
        | db    | -0.0138    | 0.0138     | 0.0000     | 0.0138
----------------------------------------------------------------
```

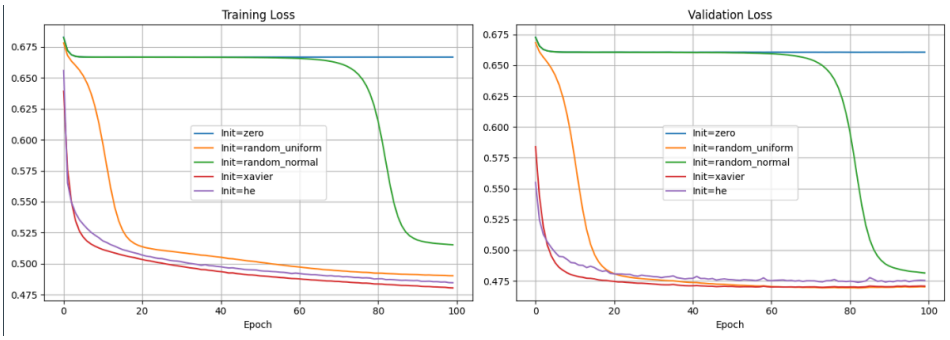


</td>
</tr>
</table>

Dari segi distribusi bobot, LR=0.1 menghasilkan bobot dengan rentang dan *standard deviation* yang lebih besar, mencerminkan *update* yang agresif. LR=0.01 dan LR=0.001 menghasilkan bobot lebih terpusat di sekitar nol. Dari sisi gradien, ketiga LR menghasilkan *magnitude* yang relatif sebanding. LR=0.001 memiliki *standard deviation* gradien tertinggi di *output layer* (0.0257), kemungkinan besar karena model belum konvergen sehingga *error* di *output layer* masih besar. LR=0.01 memiliki gradien paling stabil dan merata antar-*layer*. Kesimpulan singkatnya, LR=0.01 adalah pilihan optimal karena memiliki nilai akurasi tertinggi, kurva *loss* yang stabil, dan distribusi gradien paling merata.

v. Pengaruh Inisialisasi Bobot

Pengujian dilakukan dengan 5 variasi inisiasi bobot, yaitu zero, random uniform, random normal, xavier, dan he. Pengujian dilakukan menggunakan arsitektur [11, 64, 32, 2] dengan kombinasi fungsi aktivasi ReLU-ReLU-Softmax, 100 epoch, learning rate 0.01, dan *batch size* 32.



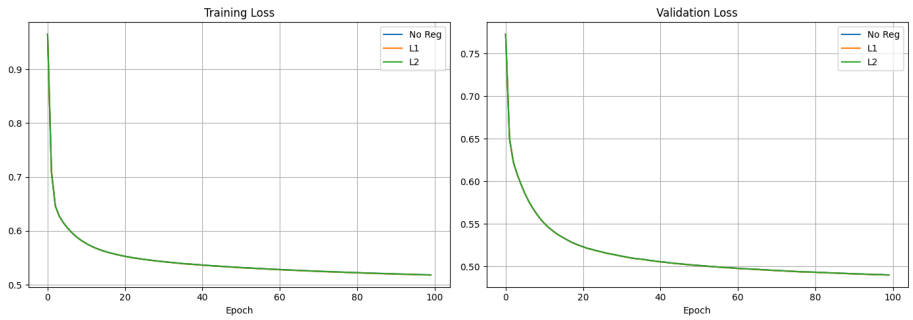
Dapat dilihat dari grafik *loss* di atas, Init=xavier dan Init=he konvergen paling cepat dan mencapai training loss terendah sekitar 0.48, serta menunjukkan konvergensi yang stabil dengan validation loss yang rendah dan konsisten juga. Init=zero gagal belajar sama sekali dengan nilai loss yang stagnan mendatar sejak awal, mengindikasikan model tidak dapat memperbarui bobotnya. Init=random_normal konvergen sangat lambat dengan penurunan yang baru terjadi setelah epoch 70-an, dan training loss masih terus turun di epoch 100, mengindikasikan model belum konvergen sepenuhnya dalam 100 epoch. Init=random_uniform agak telat mencapai training loss rendah, namun berhasil mengikuti di belakang xavier dan he setelah epoch ~18.

```
Init Method=zero | Test Accuracy: 0.6125
Init Method=random_uniform | Test Accuracy: 0.7610
Init Method=random_normal | Test Accuracy: 0.7565
Init Method=xavier | Test Accuracy: 0.7600
Init Method=he | Test Accuracy: 0.7520
```

Init=random_uniform memiliki akurasi yang tertinggi sebesar 0.7610 diikuti dengan Init=xavier dengan nilai 0.7600. Init=random_normal dan he memiliki akurasi menengah di sekitaran 0.75. Init=zero memiliki akurasi terendah dengan nilai 0.6125 karena kemungkinan kegagalan perbaruan bobot pada analisis sebelumnya.

vi. Pengaruh Regularisasi

Pengujian dilakukan dengan 3 variasi kondisi, yaitu tanpa regularisasi, L1 (lambda=0.0001), dan L2 (lambda=0.0001). Pengujian dilakukan menggunakan arsitektur tetap [11, 64, 32, 2] dengan kombinasi fungsi aktivasi ReLU-ReLU-Softmax, inisialisasi dengan metode He, 100 epoch, dan *batch size* 32.



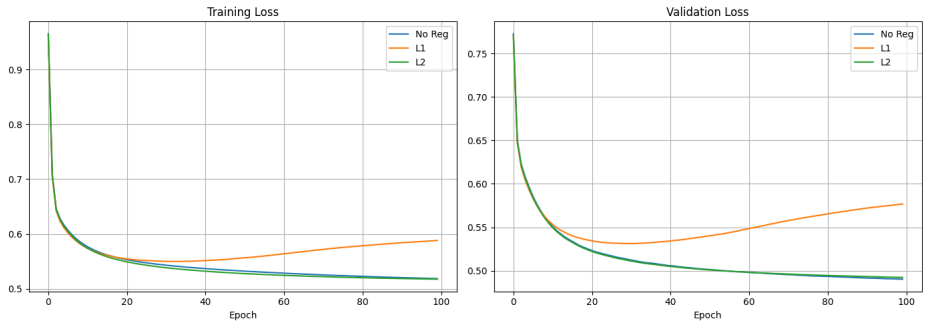
Ketiga kurva *training loss* dan *validation loss* terlihat berhimpit sempurna sepanjang 100 *epoch*, ketiganya konvergen ke *training loss* dan *validation loss*. Hal ini menunjukkan bahwa regularisasi L1 dan L2 dengan lambda=0.0001 tidak memberikan perbedaan yang dapat teramati pada proses training maupun generalisasi model

```
No Reg      | Test Accuracy: 0.7455
L1          | Test Accuracy: 0.7440
L2          | Test Accuracy: 0.7455
```

Ketiga model menghasilkan akurasi yang hampir identik dengan model tanpa regularisasi dan L2 memilikinilai yang sama yaitu sebesar 74.55% sedangkan

L1 sedikit lebih rendah dengan nilai sebesar 74.40%. Perbedaan yang sangat kecil ini wajar mengingat lambda=0.0001 yang digunakan sangat kecil.

Lalu, jika dilakukan pengujian dengan nilai lambda sebesar 0,01, dihasilkan kruva *loss* dan akurasi seperti berikut.



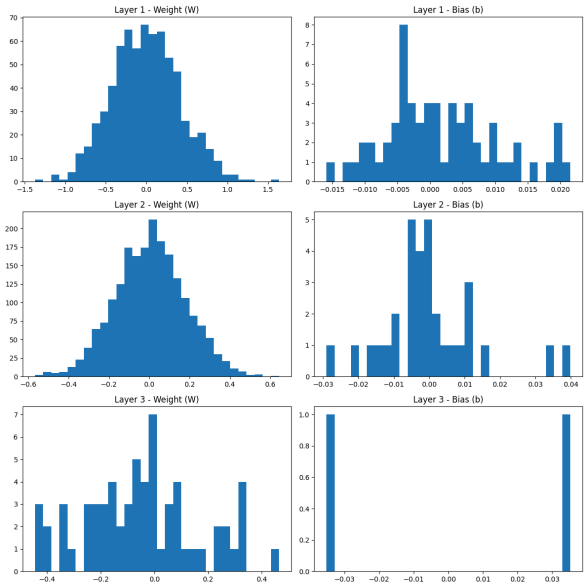

```
No Reg      | Test Accuracy: 0.7455
L1          | Test Accuracy: 0.7135
L2          | Test Accuracy: 0.7400
```

Dengan nilai lambda=0,01,perbedaan antarkondisi lebih terlihat jelas. L1 mengalami kenaikan pada *training loss* dan *validation loss* setelah sekitar *epoch* 30.Hal ini menunjukkan bahwa penalti L1 yang terlalu besar hingga menghambat model untuk *fit* terhadap data. Sedangkan model tanpa regularisasi dan L2 tetap konvergen stabil ke nilai *loss* sekitar 0.5. Dari segi akurasi, L1 turun secara signifikan menjadi 71,35% dan L2 turun sedikit menjadi 74,00%. Hal ini menunjukkan bahwa nilai lambda yang terlalu besar pada L1 dapat merugikan performa model, sementara L2 lebih *robust* terhadap kenaikan nilai lambda karena penaltinya bersifat *smooth*.

Kembali ke model dengan nilai lambda sebesar 0,001, berikut distribusi bobot dan gradien pada ketiga kondisi.

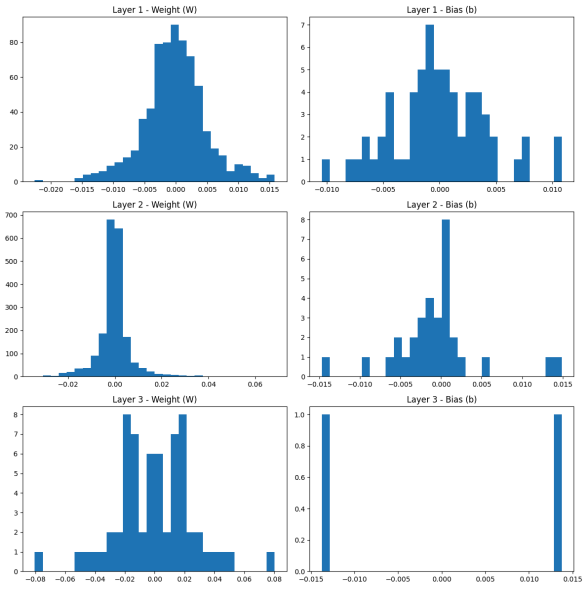
Learning Rate	**Komponen Pengujian**	**Hasil Uji**
Tanpa regularisasi	Distribusi bobot dan bias	

```
================================================================
                DISTRIBUSI BOBOT (WEIGHTS) & BIAS
================================================================
Layer   | Param | Min        | Max        | Mean       | Std
----------------------------------------------------------------
1       | W     | -1.3731    | 1.6344     | -0.0045    | 0.4193
        | b     | -0.0159    | 0.0215     | 0.0017     | 0.0086
----------------------------------------------------------------
2       | W     | -0.5667    | 0.6435     | 0.0071     | 0.1745
        | b     | -0.0291    | 0.0399     | -0.0000    | 0.0135
----------------------------------------------------------------
3       | W     | -0.4448    | 0.4638     | -0.0478    | 0.2102
        | b     | -0.0353    | 0.0353     | -0.0000    | 0.0353
----------------------------------------------------------------
```

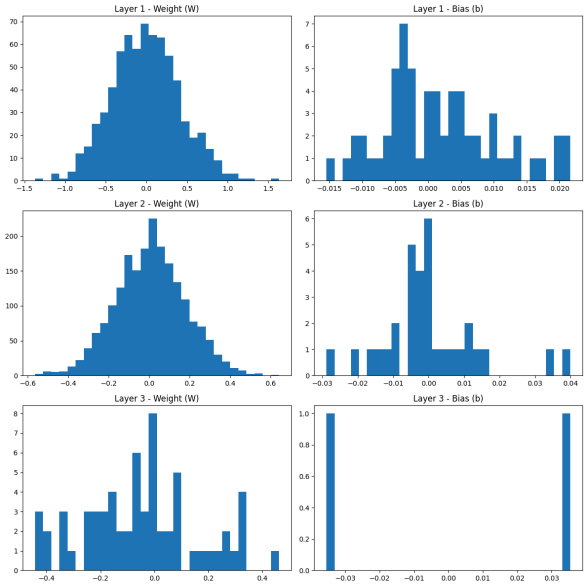


Distribusi gradien

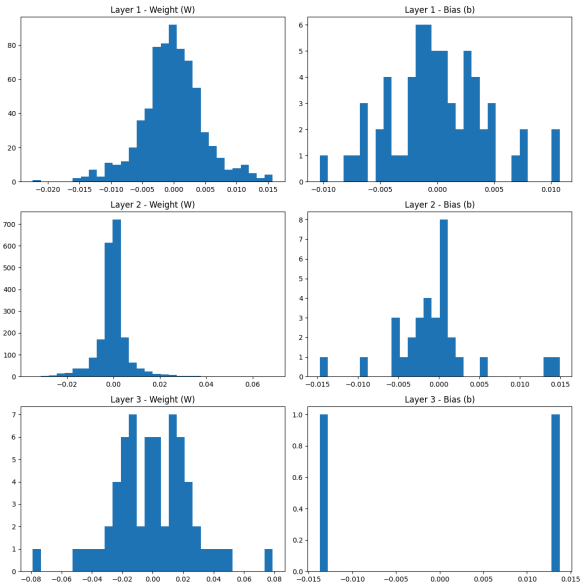
```
================================================================
                DISTRIBUSI GRADIEN (ERROR SIGNAL)
================================================================
Layer   | Param | Min        | Max        | Mean       | Std
----------------------------------------------------------------
1       | dW    | -0.0225    | 0.0158     | -0.0002    | 0.0050
        | db    | -0.0105    | 0.0108     | -0.0000    | 0.0042
----------------------------------------------------------------
2       | dW    | -0.0343    | 0.0684     | -0.0003    | 0.0071
        | db    | -0.0147    | 0.0149     | -0.0006    | 0.0053
----------------------------------------------------------------
3       | dW    | -0.0805    | 0.0805     | 0.0000     | 0.0257
        | db    | -0.0138    | 0.0138     | 0.0000     | 0.0138
----------------------------------------------------------------
```

<table>
<tr>
<td></td>
<td></td>
<td> 
</td>
</tr>
<tr>
<td>L1 dengan lambda=0,0001</td>
<td>Distribusi bobot dan bias</td>
<td>

```
==========================================================
              DISTRIBUSI BOBOT (WEIGHTS) & BIAS
==========================================================
Layer   | Param | Min        | Max        | Mean       | Std
----------------------------------------------------------
1       | W     | -1.3707    | 1.6321     | -0.0045    | 0.4176
        | b     | -0.0155    | 0.0216     | 0.0018     | 0.0086
----------------------------------------------------------
2       | W     | -0.5645    | 0.6412     | 0.0071     | 0.1728
        | b     | -0.0289    | 0.0400     | 0.0000     | 0.0135
----------------------------------------------------------
3       | W     | -0.4434    | 0.4626     | -0.0474    | 0.2089
        | b     | -0.0355    | 0.0355     | -0.0000    | 0.0355
----------------------------------------------------------
```

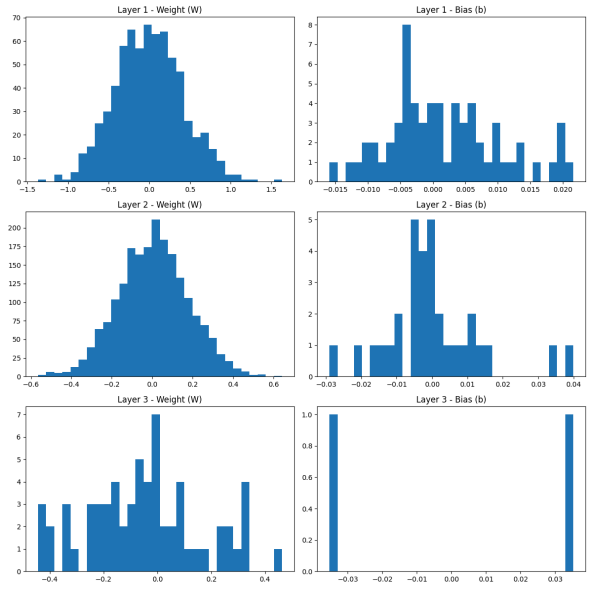

</td>
</tr>
</table>

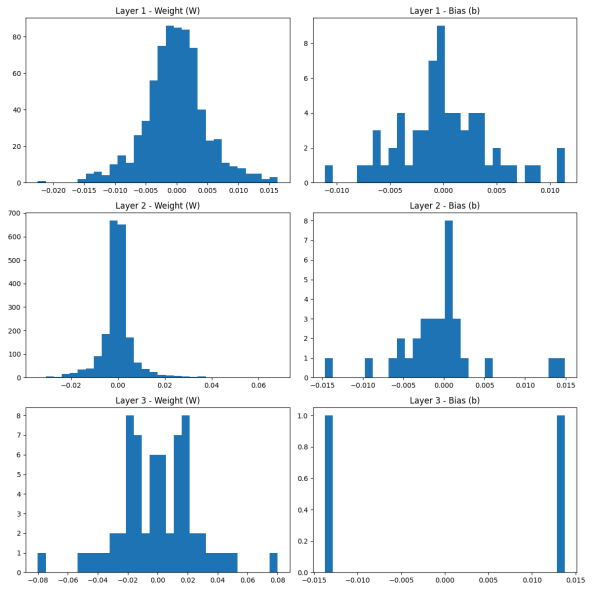
<table>
<tr>
<td></td>
<td>Distribusi gradien</td>
<td>
<pre>
==========
DISTRIBUSI GRADIEN (ERROR SIGNAL)
==========
Layer | Param | Min | Max | Mean | Std
---
1 | dW | -0.0224 | 0.0157 | -0.0002 | 0.0050
| db | -0.0103 | 0.0108 | 0.0000 | 0.0042
---
2 | dW | -0.0347 | 0.0686 | -0.0003 | 0.0071
| db | -0.0147 | 0.0149 | -0.0006 | 0.0053
---
3 | dW | -0.0789 | 0.0789 | -0.0000 | 0.0252
| db | -0.0137 | 0.0137 | -0.0000 | 0.0137

</pre>


</td>
</tr>
<tr>
<td>L2 dengan lambda=0,0001</td>
<td>Distribusi bobot dan bias</td>
<td>
<pre>
==========
DISTRIBUSI BOBOT (WEIGHTS) & BIAS
==========
Layer | Param | Min | Max | Mean | Std
---
1 | W | -1.3700 | 1.6307 | -0.0045 | 0.4184
| b | -0.0159 | 0.0216 | 0.0017 | 0.0086
---
2 | W | -0.5654 | 0.6421 | 0.0071 | 0.1741
| b | -0.0290 | 0.0400 | -0.0000 | 0.0135
---
3 | W | -0.4439 | 0.4632 | -0.0477 | 0.2098
| b | -0.0353 | 0.0353 | -0.0000 | 0.0353

</pre>
</td>
</tr>
</table>

<table>
<tr>
<td></td>
<td></td>
<td> 
</td>
</tr>
<tr>
<td></td>
<td>Distribusi gradien</td>
<td>
<pre>
==========
DISTRIBUSI GRADIEN (ERROR SIGNAL)
==========
Layer | Param | Min | Max | Mean | Std
----
1 | dW | -0.0225 | 0.0164 | -0.0002 | 0.0050
| db | -0.0111 | 0.0114 | 0.0000 | 0.0042
----
2 | dW | -0.0342 | 0.0682 | -0.0003 | 0.0071
| db | -0.0147 | 0.0149 | -0.0006 | 0.0053
----
3 | dW | -0.0801 | 0.0801 | -0.0000 | 0.0256
| db | -0.0138 | 0.0138 | 0.0000 | 0.0138

</pre>

</td>
</tr>
</table>

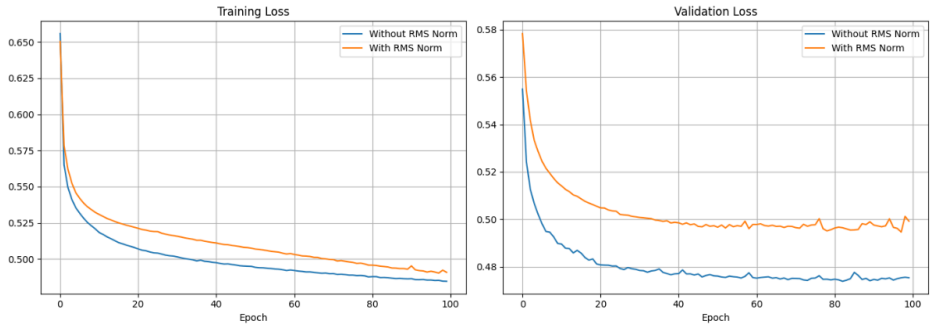
Distribusi bobot L1 memiliki *standard deviation* sedikit lebih kecil (0.4176) dibandingkan dengan yang tanpa regularisasi (0.4193) dan L2 (0.4184). Hal ini

konsisten dengan L1 yang lebih mendorong *sparsity.* L2 menghasilkan bobot dengan *magnitude* sedikit lebih kecil secara keseluruhan, namun distribusinya tetap *smooth*. Perbedaan distribusi gradien di antara ketiga kondisi sangat minimal diakibatkan oleh kontribusi penalti dengan nilai lambda sebesar 0,0001 yang hampir tidak terasa.

Kesimpulannya, pada dataset ini, regularisasi tidak memberikan dampak positif yang signifikan. Dengan menggunakan lambda=0.0001, model dengan L1 dan L2 menghasilkan performa yang hampir identik dengan model tanpa regularisasi. Jika menggunakan lambda=0.01, performa model dengan regularisasi justru menurun, terutama pada L1 (71.35%). Hal ini menunjukkan bahwa model tidak mengalami *overfitting* yang berarti, sehingga penambahan regularisasi dengan lambda yang lebih besar justru menghambat proses pembelajaran.

vii. Pengaruh Normalisasi RMSNorm

Pengujian dilakukan dengan 2 variasi, yaitu yang satu menggunakan RMSNorm dan yang tidak menggunakan. Pengujian dilakukan menggunakan arsitektur [11, 64, 32, 2] dengan kombinasi fungsi aktivasi ReLU-ReLU-Softmax, inisiasi he, 100 epoch, learning rate 0.01, dan *batch size* 32.



Dapat dilihat dari grafik *loss* di atas, Without RMS Norm konvergen lebih cepat dan mencapai training loss lebih rendah, serta menunjukkan konvergensi yang stabil dengan validation loss yang sangat konsisten hingga akhir epoch. Sebaliknya, With RMS Norm konvergen sedikit lebih lambat dengan nilai training loss yang lebih tinggi . Meskipun validation lossnya cukup stabil dan tidak menunjukkan tanda-tanda overfitting yang jelas, nilainya tertahan di angka yang lebih tinggi dan sedikit lebih fluktuatif. Ini menunjukkan bahwa penambahan teknik normalisasi ini pada arsitektur tersebut justru kurang optimal dan sedikit menghambat model untuk mencapai performa terbaiknya.

```
Without RMS Norm | Test Accuracy: 0.7520
With RMS Norm | Test Accuracy: 0.7490
```

Sesuai dengan loss sebelumnya, Without RMS Norm memiliki akurasi yang lebih tinggi sebesar 0.752. Sedangkan, RMS Norm sedikit lebih kecil dengan nilai 0.7490

viii. Perbandingan dengan *Library* sklearn

Pengujian dilakukan menggunakan FFNN custom dan MLP dari library sklearn. Pengujian dilakukan menggunakan arsitektur [11, 64, 32, 2] dengan kombinasi fungsi aktivasi ReLU-ReLU-Softmax, inisiasi he, 100 epoch, learning rate 0.01, solver adam, dan *batch size* 32.

```
================================================================
PERBANDINGAN MODEL
================================================================

Custom FFNN Accuracy: 0.6125
sklearn MLPClassifier Accuracy: 0.7200

================================================================
SKLEARN MODEL - CLASSIFICATION REPORT
================================================================
              precision    recall  f1-score   support

           0       0.68      0.53      0.59       775
           1       0.74      0.84      0.79      1225

    accuracy                           0.72      2000
   macro avg       0.71      0.68      0.69      2000
weighted avg       0.71      0.72      0.71      2000
```

Sklearn MLPClassifier menghasilkan performa yang jauh lebih baik dengan akurasi tes 72%, dibandingkan dengan Custom FFNN yang hanya mencapai akurasi 61.25% meskipun menggunakan konfigurasi hyperparameter yang sama persis. Perbedaan performa yang cukup signifikan ini menandakan bahwa implementasi dari skicit memiliki optimasi tingkat lanjut yang belum terakomodasi sepenuhnya pada model FFNN custom.

C. Kesimpulan dan Saran

Dari tugas ini, kami sudah membangun model Feedforward Neural Network (FFNN) dari nol. Model ini lengkap dengan fitur forward propagation dan backward propagation. Setelah menguji model ini menggunakan dataset yang sudah diberikan, kami bisa menyimpulkan

bahwa model ini performanya dapat dipengaruhi dari perubahan arsitektur, fungsi aktivasi yang digunakan, inisialisasi bobot, learning rate, regularisasi, dan RMS normalisasi. Kami juga sudah membandingkan model yang telah dibuat dengan model dari sklearn dan menyimpulkan bahwa model kami sudah cukup bagus dalam performa dan bisa mendekati model dari library populer ini.

Beberapa saran untuk kedepannya:

1. Kode masih bisa dibuat lebih efisien lagi supaya komputasi perhitungan matriks bisa lebih cepat.
2. Pemilihan hyperparameter yang lebih cocok untuk lebih menunjukkan pengaruh variabel bebas saat pengujian

D. Pembagian Tugas Tiap Anggota Kelompok

Berikut pembagian tugas pada kelompok kami:

NIM	Nama	Tugas
13523034	Rafizan Muhammad Syawalazmi	- Implementasi RMS normalisasi - Implementasi utils functions - Implementasi visualisasi distribusi bobot, bias, dan gradien - Membuat EDA data - Membuat testing - Laporan (kesimpulan, pengujian width, depth, fungsi aktivasi, inisiasi bobot, RMSNorm, dan sklearn)
13523056	Salman Hanif	- Pembuatan Struktur dasar kelas FFNN - Implementasi Automatic Differentiation - Implementasi Forward propagation - Implementasi Fungsi aktivasi tambahan - Laporan (implementasi kelas, Forward propagation)
13523058	Noumisyifa Nabila Nareswari	- Implementasi semua *loss functions* - Implementasi *backward propagation* - Implementasi mekanisme *save* & *load* model - Implementasi Adam *optimizer* - Laporan (deskripsi permasalahan, *backward propagation*, pengujian *learning rate*, pengujian *regularization*)

E. Referensi

Das, A. (2025, July 12). Leaky Relu Activation Function in Deep Learning. GeeksforGeeks. Retrieved 2026, from www.geeksforgeeks.org/machine-learning/leaky-relu-activation-function-in-deep-learning/

Kingma, D., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. Arxiv. Retrieved 2026, from arxiv.org/abs/1412.6980

Mahajan, R. (2025, July 23). Kaiming Initialization in Deep Learning. GeeksforGeeks. Retrieved 2026, from www.geeksforgeeks.org/deep-learning/kaiming-initialization-in-deep-learning/

Mitchell, T. M. (1997). Machine Learning. McGraw-Hill Education.

Paul, S. (2025, July 23). Xavier initialization. GeeksforGeeks. Retrieved 2026, from www.geeksforgeeks.org/deep-learning/xavier-initialization/

Ramachandran, P., Zoph, B., & Le, Q. (n.d.). Swish: A Self-Gated Activation Function. Arxiv. Retrieved 2026, from arxiv.org/pdf/1710.05941v1

Raschka, S., Liu, Y., Mirjalili, V., & Dzhulgakov, D. (2022). Machine Learning with PyTorch and Scikit-Learn: Develop Machine Learning and Deep Learning Models with Python. Packt Publishing.

Starmer, J. (2019). Gradient Descent, Step-by-Step. Youtube. youtu.be/sDv4f4s2SB8?si=smKTvmfYoRS50FwZ

Zhang, B., & Sennrich, R. (2019). Root Mean Square Layer Normalization. Arxiv. Retrieved 2026, from arxiv.org/abs/1910.07467

F. Lampiran Form Penggunaan AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM :

1. Rafizan Muhammad Syawalazmi / 13523034
2. Salman Hanif / 13523056
3. Noumisyifa Nabila Nareswari / 13523058

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 1 IF3270 Pembelajaran Mesin FeedForward Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1 - No AI
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2 - AI-assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2 - AI-assisted idea generation and structuring
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1 - No AI
Pembuatan Kode: menggunakan tools seperti ChatGPT, Gemini, Copilot, atau sejenisnya.	Ya	2 - AI-assisted idea generation and structuring

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 15 Maret 2026

Rafizan Muhammad S.	Salman Hanif	Noumisyifa Nabila N.
13523034	13523056	13523058